Xeno

The core workflow is clear: AI converts natural-language intent into structured tools, segmentation materializes audience membership, the campaign engine personalizes and dispatches messages, and receipts update a forward-only engagement funnel. I'm checking the receipt callback service and UI routes next, because those explain the project's strongest engineering choices and the user-visible feature set.

The remaining pieces show a deliberate demo architecture rather than a production messaging provider: the channel service accepts work immediately and sends simulated callbacks, while the CRM protects metrics with idempotent status transitions. I'm doing one final pass over authentication, tests, and the simulator so I can distinguish implemented behavior from planned or simulated behavior in the final overview.

# XENO CRM Project Overview

XENO is an **AI-native customer relationship management and marketing automation platform** built for a coffee brand called **BeanBox Coffee**.

The main idea is:

> A marketer can describe a campaign in natural language, and the AI can find customers, create a segment, draft a message, create a campaign, send it, and show engagement results.

Example:

> "Find customers who spent more than ₹5,000 and send them a WhatsApp campaign with 20% off cold brew."

The system can turn that request into structured operations automatically.

## Technology Stack

### 1. Frontend: Next.js, React, TypeScript

Used technologies:

- Next.js 16
- React 19

- TypeScript
- Next.js App Router
- Custom Vanilla CSS
- Vercel deployment

The frontend contains:

- Dashboard
- AI Assistant chat
- Customer management
- Customer search
- Segment management
- Campaign management
- Data import
- Login and registration screens
- Dark/light theme support
- Responsive mobile layout

Main frontend files:

- `page.tsx`
- `page.tsx`
- `page.tsx`
- `api.ts`

### Why Next.js?

Next.js was selected because:

- It is built on React and provides a structured application framework.
- The App Router makes it easy to organize dashboard routes.
- It provides good production performance and deployment support.
- It works naturally with Vercel.
- It supports TypeScript and modern React features.
- It is more scalable than building everything with plain React or static HTML.

### Why React?

React is useful because the application contains many dynamic elements:

- Chat messages
- Loading states
- Campaign statistics
- Search results
- Modals
- Forms
- Pagination
- Theme changes
- Responsive sidebar behavior

React makes these interactive UI updates easier to manage than manually changing the DOM.

### Why Vanilla CSS?

The project uses custom CSS instead of a large UI framework because:

- The application needed a specific visual style.
- It uses a custom glassmorphism interface.
- There was no need to add the bundle size or constraints of a large component library.
- It provides full control over colors, layouts, animations, responsive behavior, and themes.

---

## 2. Backend: FastAPI and Python

Used technologies:

- Python 3.12
- FastAPI
- Uvicorn
- Pydantic
- SQLAlchemy 2.0
- AsyncSQLAlchemy
- `asyncpg`
- `aiosqlite`
- `httpx`

The backend is the main application service. It provides REST APIs for:

- Customers
- Orders
- Segments
- Campaigns
- AI chat
- Delivery receipts
- Customer statistics
- Data ingestion

Main backend files:

- `main.py`
- `ai_agent.py`
- `segmentation.py`
- `campaign_engine.py`
- `ingestion.py`

### Why FastAPI?

FastAPI was selected because:

- It is fast and lightweight.
- It supports asynchronous endpoints naturally.
- It automatically generates OpenAPI documentation.
- Pydantic provides request and response validation.
- It works well with AI APIs and external HTTP services.
- It is simpler and less heavy than Django for an API-focused system.

Django could have been used, but it would add more framework features than this project needs. The application mainly requires APIs, database operations, AI orchestration, and service-to-service communication.

### Why asynchronous Python?

The application performs many I/O operations:

- Database queries
- AI model calls
- HTTP requests to the channel service
- Receipt callbacks

Asynchronous code allows the backend to handle these operations efficiently without blocking the server unnecessarily.

---

## 3. Database: PostgreSQL and Supabase

The production database is:

- PostgreSQL
- Hosted using Supabase

The project also supports SQLite locally through SQLAlchemy and `aiosqlite`.

The schema is defined in `schema.sql`.

### Main database tables

#### `customers`

Stores shopper information:

- Name
- Email
- Phone
- Total spending
- Number of orders
- Average order value
- First order date
- Last order date
- Tags

#### `orders`

Stores purchase history:

- Customer
- Order amount
- Order items

- Order channel
- Order status
- Creation date

#### `segments`

Stores reusable customer audiences:

- Segment name
- Segment rules
- Customer count
- Natural-language description
- Whether it was created by AI

#### `segment_members`

Stores the customers belonging to each segment.

This is a many-to-many relationship between customers and segments.

#### `campaigns`

Stores marketing campaign information:

- Campaign name
- Target segment
- Message template
- Channel
- Campaign status
- Delivery counts
- Open counts
- Click counts
- Conversion counts

#### `communications`

Stores one communication per recipient.

This provides individual delivery tracking for every customer.

#### `ai_conversations`

Stores AI chat history and conversation context.

### Why PostgreSQL?

PostgreSQL was selected because:

- It is reliable and production-ready.
- It supports strong relational data modeling.
- It supports JSON and JSONB fields for flexible data such as tags, order items, and segment rules.
- It provides indexing and aggregation capabilities.
- Supabase provides hosted PostgreSQL with a convenient management interface.

MongoDB could have been used for flexible JSON data, but the project has strong relationships between customers, orders, segments, campaigns, and communications. PostgreSQL is better suited for these relational queries and reporting requirements.

### Why SQLAlchemy?

SQLAlchemy was selected because:

- It provides database abstraction.
- It supports both PostgreSQL and SQLite.
- It supports asynchronous database access.
- It avoids writing database-specific logic throughout the application.
- It makes models, queries, relationships, and transactions easier to maintain.

The ingestion service uses PostgreSQL and SQLite-specific upsert operations where necessary, while keeping most business logic portable.

---

# AI System

The AI system uses:

- Groq
- Llama 3.3 70B

- Google Gemini 2.5 Flash as fallback

The AI code is mainly in `ai_agent.py`.

## Why Groq?

Groq was selected as the primary provider because:

- It has very fast inference.
- Fast responses are important for a chat-first CRM.
- It supports function/tool calling.
- Llama models are capable of understanding business instructions and customer segmentation requests.

## Why Gemini as fallback?

Gemini is used as a backup provider because:

- AI APIs can experience rate limits or temporary failures.
- A fallback improves availability.
- The user can still receive an AI response if Groq is unavailable.

This is a dual-provider architecture:

```text
User message
    |
    v
Groq / Llama
    |
    | if unavailable
    v
Gemini Flash
```

## AI tools

The AI can use tools such as:

- `query_customers`

- `create_segment`
- `draft_message`
- `create_campaign`
- `send_campaign`
- `get_campaign_stats`
- `get_insights`

This makes the AI more than a chatbot. It can interact with the application and perform business operations.

---

# Customer Segmentation

The segmentation engine is implemented in `segmentation.py`.

Segments use structured rules such as:

```json
[
  {
    "field": "total_spend",
    "operator": ">",
    "value": 5000
  }
]
```

Supported filtering examples include:

- Total spend greater than a value
- Order count
- Name or email search
- Customers inactive for a number of days
- Customers active recently
- Tags
- Null and non-null values
- Multiple rules combined with `AND`

## Why materialized segments?

When a segment is created, matching customer IDs are stored in `segment_members`.

This was chosen because:

- Campaign sending becomes faster.
- The audience is clearly recorded.
- The system does not need to re-evaluate every rule during campaign dispatch.
- The exact audience used by a campaign can be audited.

The tradeoff is that segment membership may need to be refreshed when customer data changes.

---

# Campaign System

The campaign engine is implemented in `campaign_engine.py`.

Campaign workflow:

1. Select a target segment.
2. Load all customers in that segment.
3. Personalize the message for each customer.
4. Create a communication record for every recipient.
5. Send the batch to the channel service.
6. Receive delivery callbacks.
7. Update campaign statistics.

Supported channels include:

- WhatsApp
- SMS
- Email
- RCS

Messages support placeholders such as:

```text
Hello {{first_name}}, you have spent {{total_spend}} with BeanBox.
```

```

These are replaced for every customer before sending.

---

# Separate Channel Service

The channel service is another FastAPI application:

- `main.py`
- `simulator.py`

It accepts a batch of communications and returns immediately with an accepted response.

It then simulates:

```text
sent
  |
  +--> delivered
         |
         +--> opened
                |
                +--> read
                       |
                       +--> clicked
                              |
                              +--> converted
```

The simulator randomly generates:

- Delivery failures
- Opens
- Reads
- Clicks
- Conversions
```

## Why create a separate service?

A real marketing system would normally communicate with external providers such as:

- WhatsApp Business API
- Twilio
- SendGrid
- Mailchimp
- Firebase
- Other SMS or email providers

For this project, a separate simulator demonstrates the same architecture without requiring real messaging accounts or sending real messages.

It also shows important distributed-system concepts:

- Service-to-service HTTP communication
- Asynchronous background work
- Webhook/callback processing
- Retry handling
- Delivery status tracking
- Failure handling

---

# Receipt and Conversion Tracking

The CRM receives callbacks at:

```text
/api/receipts/batch
```

The receipt handler prevents invalid status changes.

For example:

- A communication cannot go from `clicked` back to `opened`.
- A delivered communication cannot later become failed.
- Duplicate callbacks do not inflate campaign counters.

This is implemented using status ordering and idempotency logic in `receipts.py`.

Campaign metrics include:

- Total recipients
- Sent
- Delivered
- Failed
- Opened
- Read
- Clicked
- Converted
- Delivery rate
- Open rate
- Click rate
- Conversion rate

Conversion tracking represents a customer placing an order because of a campaign.

---

# Data Import and Seed Data

The project includes:

- Customer seed data
- Order seed data
- Import scripts
- Bulk ingestion logic

The ingestion service supports:

- Bulk inserts
- Upserts
- UUID conversion
- ISO date conversion
- Customer lookup by email
- Order-to-customer linking
- Aggregate recalculation

It also recalculates:

- Total spend
- Order count
- Average order value
- First order date
- Last order date

This is much faster than inserting records one at a time.

---

# Frontend Features

The dashboard provides:

- Total customers
- Total revenue
- Average order value
- Campaign overview
- Quick actions
- AI assistant access

The customer page provides:

- Customer search
- Pagination
- Customer statistics
- Add customer functionality
- Tags and purchase information

The AI chat page provides:

- Natural-language queries
- Conversation history
- Suggested prompts
- Loading indicators
- Display of AI actions
- Campaign and segmentation operations

Other pages include:

- Segments
- Campaigns
- Data import
- Login
- Registration

---

# Authentication

The current authentication is demo-level.

User details are stored in browser `localStorage` through `auth.ts`.

This is suitable for a prototype demonstration, but it is not production authentication because:

- There is no backend session validation.
- There are no secure cookies.
- There is no password verification service.
- Users can technically modify local storage.

A production version should use:

- JWT or secure session cookies
- Password hashing
- Backend authorization
- Role-based access control
- Proper user and organization tables

---

# Deployment

The deployment design is:

```text
```

Frontend -> Vercel
Backend -> Render
Channel API -> Render
Database -> Supabase PostgreSQL
AI providers -> Groq and Gemini
```

Deployment configuration is in `render.yaml`.

The frontend communicates with the backend through `NEXT_PUBLIC_API_URL`.

---

# Testing

The backend includes tests for:

- AI intent detection
- Campaign creation intent
- Campaign launch intent
- Database engine configuration
- PostgreSQL async settings
- SQLite compatibility

Relevant files:

- `test_ai_intents.py`
- `test_database.py`

The frontend includes linting and production build scripts through `package.json`.

---

# Why This Overall Architecture?

The architecture was designed around the project’s main requirements:

- Natural-language marketing workflow
- Fast customer filtering
- Async AI and HTTP operations
```

- Relational customer and campaign data
- Auditable campaign delivery
- Simulated real-world messaging
- Deployment as separate scalable services

The project is intentionally more than a CRUD dashboard. Its main differentiator is the AI workflow:

```text
Natural-language request
        |
        v
AI understands intent
        |
        v
Structured customer query
        |
        v
Segment creation
        |
        v
Message generation
        |
        v
Campaign creation
        |
        v
Campaign dispatch
        |
        v
Delivery and conversion analytics
```

In summary, XENO is a **chat-first AI marketing CRM** that combines customer data management, intelligent segmentation, personalized campaign creation, simulated omnichannel delivery, and conversion analytics in one system.
ReportMaster AI — Financial Reporting Intelligence Hub

Associated with Course Work

Domain: Financial reporting automation / Accounting knowledge management / Enterprise AI

Built a finance-focused RAG solution that transforms internal accounting manuals into an intelligent assistant. I developed the full system pipeline: document ingestion, sentence-transformer embeddings, FAISS vector search, and Google Gemini-backed answer generation. The project delivers fast, grounded responses with source citations so reporting and audit teams can ask complex accounting questions and get accurate, context-aware guidance from internal policy documents.

RentWise – Rental Price Fairness Platform

The implementation confirms the product has two distinct roles and workflows: renters publish/manage listings, while customers search, save, contact owners, review, and request pricing analysis. The backend also stores images in Cloudflare R2-compatible object storage rather than the relational database, which is an important design decision to explain.

# RentWise Project Explanation

## 1. One-Minute Project Introduction

**RentWise is a full-stack rental property platform that helps users discover rental properties and determine whether a property's rent is fair, overpriced, or underpriced.**

The system has two user roles:

- **Customer:** searches properties, filters listings, views details, saves favorites, contacts owners, writes reviews, and analyzes rental prices.
- **Renter:** creates, edits, uploads images for, and deletes property listings.

The main differentiating feature is the **rental price analysis system**, which uses a machine-learning regression model to estimate the expected market rent based on:

- City
- Locality
- Number of bedrooms
- Property area in square feet

The application compares the predicted rent with the owner’s actual rent and displays:

- Predicted market rent
- Difference from the listed rent
- Whether it is overpriced, underpriced, or fairly priced
- A market range around the predicted value

---

# 2. Technology Stack

## Frontend

### React

React is used to build the user interface with reusable components and page-based views.

Examples:

- Navbar
- Protected routes
- Property cards
- Login and registration forms
- Property upload forms
- Wishlist
- Property details

**Why React?**

- Component-based architecture
- Good for interactive applications
- Easy state management with hooks such as `useState` and `useEffect`
- Large ecosystem
- Suitable for single-page applications

**Why not plain HTML, CSS, and JavaScript?**

A rental platform has many dynamic interactions: filtering, login state, wishlist updates, image previews, pagination, and routing. React makes these interactions easier to organize and maintain.

### Vite

Vite is used as the frontend build tool.

**Why Vite?**

- Very fast development server
- Fast hot module replacement
- Simple configuration
- Faster startup than traditional bundlers such as older Create React App setups

### React Router

React Router handles client-side navigation.

Routes include:

- `/`
- `/explore`
- `/login`
- `/register`
- `/add`
- `/wishlist`
- `/my-listings`
- `/edit/:id`
- `/property/:id`

**Why React Router?**

It allows the application to behave like a single-page application without reloading the browser for every page.

### Axios

Axios is used for communication between the React frontend and Flask backend.

It also has an interceptor that automatically attaches the JWT token:

```text
Authorization: Bearer <token>
```

**Why Axios instead of `fetch`?**

Axios provides:

- Cleaner API syntax
- Automatic JSON handling
- Request and response interceptors
- Easier error handling
- Centralized API configuration

The implementation is in `axios.js`.

### Tailwind CSS

Tailwind is used for styling.

**Why Tailwind?**

- Utility-first styling
- Fast development
- Responsive design classes
- Dark-mode support

- Avoids writing large separate CSS files

The project supports light and dark themes using Tailwind’s class-based dark mode.

### Framer Motion

Framer Motion is included as a frontend dependency for animations. It is available for UI transitions and motion effects, although it is not the main part of the current business logic.

---

## Backend

### Python

Python is used for the backend and machine-learning functionality.

**Why Python?**

- Excellent web-development frameworks
- Strong data-science ecosystem
- Easy integration with pandas, NumPy, and scikit-learn
- Suitable for both API development and machine learning

### Flask

Flask is used to build the REST API.

**Why Flask?**

- Lightweight and flexible
- Easy to create REST endpoints
- Good integration with SQLAlchemy and JWT
- Suitable for a focused backend application

- Less boilerplate than larger frameworks

**Why not Django?**

Django provides more built-in features, but RentWise only needs a focused REST API. Flask gives more control and keeps the backend lightweight.

The application factory is defined in `__init__.py`.

### Flask-SQLAlchemy and SQLAlchemy

SQLAlchemy is used as the ORM.

It maps Python classes to database tables:

- `User`
- `Property`
- `PropertyImage`
- `Review`
- `Wishlist`

**Why use an ORM instead of raw SQL?**

- Safer and more maintainable database queries
- Python objects represent database entities
- Relationships are easier to manage
- Less repetitive SQL code
- Easier to change database providers

### Database

The README mentions MySQL, but the current configuration supports multiple databases:

- Production database through `DATABASE_URL`
- PostgreSQL if configured

- SQLite fallback for local development

The configuration is in `config.py`.

A strong answer would be:

> “The project is database-independent through SQLAlchemy. The original target was MySQL, while the current configuration also supports PostgreSQL through `DATABASE_URL` and SQLite as a local fallback.”

### Flask-JWT-Extended

JWT authentication is used for login sessions.

On successful login, the backend creates an access token. The frontend stores the token and sends it with protected requests.

**Why JWT?**

- Stateless authentication
- Works well with REST APIs
- Backend does not need to maintain server-side sessions
- Easy to use with separate frontend and backend deployments

**Why not traditional sessions?**

The frontend and backend are deployed separately. JWT is convenient for cross-origin API communication between a Vite frontend and Flask backend.

### Werkzeug Password Hashing

Passwords are never stored as plain text.

The `User` model uses:

```python
```

generate_password_hash(password)

check_password_hash(...)

```

This is implemented in `user.py`.

### Flask-CORS

CORS allows the deployed frontend to communicate with the Flask backend from a different domain.

The configuration is handled in `__init__.py`.

### Boto3 and Cloudflare R2

Images are uploaded to Cloudflare R2 using the S3-compatible Boto3 client.

The database stores the image URL, not the actual image file.

**Why object storage instead of storing images in the database?**

- Databases are not ideal for large binary files
- Object storage is cheaper and scalable
- Images can be served independently
- Database records remain smaller
- Better separation between application data and media files

The implementation is in:

- `r2_client.py`
- `r2_upload.py`

### Gunicorn

Gunicorn is used to run the Flask application in production.
```

The deployment command is defined in `Procfile`:

```text
gunicorn --bind :$PORT run:app
```

---

# 3. Machine-Learning System

## Problem Solved

The ML system estimates the expected monthly rent of a property.

The input features are:

- `city`
- `locality`
- `bedrooms`
- `area_sqft`

The target value is:

- `rent`

## Algorithm

The project uses a **Random Forest Regressor**.

**Why Random Forest?**

- Handles nonlinear relationships
- Works well with mixed features
- More robust than a single decision tree

- Performs well on tabular data
- Does not require feature scaling
- Can model relationships between location, area, bedrooms, and rent

**Why not linear regression?**

Rent is not always linearly related to area or bedrooms. A property in a premium locality may cost much more than a similar-sized property in another locality. Random Forest can model these nonlinear effects.

**Why not deep learning?**

The dataset is structured tabular data, not images, audio, or natural language. Deep learning would add complexity without a clear benefit for this problem.

## Data Processing

Categorical columns:

- City
- Locality

These are converted using `OneHotEncoder`.

Numerical columns:

- Bedrooms
- Area

These are passed directly to the model.

The preprocessing and model are combined into one scikit-learn `Pipeline` in `train_model.py`.

This is useful because the same preprocessing used during training is automatically applied during prediction.

## Training Data

The current training script generates synthetic rental data for cities such as:

- Hyderabad
- Chennai
- Bengaluru
- Mumbai
- Pune
- Vijayawada
- Guntur

The dataset contains approximately **14,000 records**.

The data is split into:

- 80% training data
- 20% test data

## Model Metrics

The stored metrics are:

- $R^2$ score: `0.9965`
- MAE: approximately `₹1,730`
- RMSE: approximately `₹2,092`

However, the important qualification is:

> These metrics are calculated on synthetic data generated by the project, so they should not be treated as real-world accuracy until the model is trained and validated using real rental-market data.

## Prediction Flow

1. User opens a property detail page.

2. User selects **Analyze Pricing**.
3. Frontend sends a request to:

```text
POST /analysis/<property_id>
```

4. Backend retrieves the property from the database.
5. The model predicts the expected rent.
6. The prediction is rounded to the nearest ₹100.
7. Backend compares predicted rent with the actual listed rent.
8. The result is classified:

```text
difference > ₹2,000   => Overpriced
difference < -₹2,000  => Underpriced
otherwise             => Fairly Priced
```

The active implementation is in:

- `analysis_routes.py`
- `ml_loader.py`

---

# 4. Database Design

## Users

Stores:

- ID
- Email
- Hashed password

- Role

Roles are:

- `customer`
- `renter`

## Properties

Stores:

- City
- Locality
- Bedrooms
- Area
- Rent
- Description
- Owner ID
- Creation timestamp

Each property belongs to one renter.

## Property Images

Stores:

- Image ID
- Image URL
- Property ID
- Creation timestamp

A property can have multiple images.

## Reviews

Stores:

- Rating
- Comment
- User ID
- Property ID

The backend prevents:

- Ratings outside 1 to 5
- A renter reviewing their own property
- The same user reviewing the same property more than once

## Wishlist

Stores:

- User ID
- Property ID

Customers can add and remove properties from their wishlist.

---

# 5. Main User Workflows

## Customer Workflow

1. Register as a customer.
2. Log in.
3. Search properties by city and locality.
4. Filter by bedrooms and maximum rent.
5. Sort by price or newest listings.
6. Open a property detail page.
7. View images, rent, description, and reviews.

8. Analyze the price using the ML model.
9. Add the property to the wishlist.
10. Contact the owner using the displayed email.
11. Submit a review.

## Renter Workflow

1. Register as a renter.
2. Log in.
3. Add a property.
4. Upload up to eight images.
5. View the property under My Listings.
6. Edit property details.
7. Add or delete images.
8. Delete the property when required.

## Image Upload Workflow

The frontend:

- Accepts image files only
- Limits each image to 5 MB
- Allows a maximum of 8 images
- Generates local previews
- Supports drag-and-drop

The backend:

- Receives files through multipart form data
- Generates a unique filename
- Sanitizes the original filename
- Uploads the image to R2
- Stores the resulting public URL in the database

---

# 6. API Structure

## Authentication

```text
POST /auth/register
POST /auth/login
```

## Properties

```text
POST   /properties/
GET    /properties/all
GET    /properties/filters
GET    /properties/<id>
GET    /properties/mine
PUT    /properties/<id>
DELETE /properties/<id>
```

## Images

```text
DELETE /properties/<property_id>/images/<image_id>
```

## Reviews

```text
POST /properties/<property_id>/review
```

## Wishlist

```text
POST   /properties/<property_id>/wishlist
DELETE /properties/<property_id>/wishlist
GET    /properties/wishlist
```

## Pricing Analysis

```text
POST /analysis/<property_id>
```

---

# 7. Important Architecture Explanation

The architecture is:

```text
React frontend
      |
      | Axios REST requests
      v
Flask REST API
      |
      +-- JWT authentication
      +-- SQLAlchemy ORM
      +-- PostgreSQL/MySQL/SQLite database
      +-- Cloudflare R2 image storage
      +-- scikit-learn rent prediction model
```

The frontend and backend are separated because this makes the system:

- Easier to develop independently
- Easier to deploy separately
- Suitable for web and future mobile clients
- More maintainable than putting UI and backend logic in one application

---

# 8. Security Features

Implemented security features include:

- Password hashing
- JWT authentication
- Protected backend endpoints
- Role-based frontend routing
- Owner checks before editing or deleting properties
- CORS restrictions
- Secure filenames for uploaded images
- File type and file size validation
- Prevention of duplicate reviews
- Prevention of self-reviews

For example, a renter can only edit or delete their own property.

The frontend uses `ProtectedRoute`, while the backend verifies the JWT using `@jwt_required()`.

A strong answer is:

> “Frontend route protection improves user experience, but the real security is enforced on the backend because users can bypass frontend checks. The backend verifies the token and checks ownership before modifying data.”

---

# 9. Deployment

The frontend is configured for Vercel using `vercel.json`.

The backend can run with Gunicorn using the `Procfile`.

The frontend API URL can be configured with:

```text
VITE_API_URL
```

The backend uses environment variables for:

- Database URL
- JWT secret
- Flask secret
- CORS origins
- R2 bucket name
- R2 endpoint
- R2 credentials
- R2 public URL

Secrets should not be hardcoded in the source code.

---

# 10. Why These Technologies Were Selected

| Requirement | Technology | Reason |
|---|---|---|
| Interactive UI | React | Component-based and dynamic |
| Frontend tooling | Vite | Fast development and builds |
| Routing | React Router | SPA navigation |
| API communication | Axios | Interceptors and clean requests |
| Styling | Tailwind CSS | Responsive utility classes |

| Backend API | Flask | Lightweight and flexible |

| Database access | SQLAlchemy | ORM and database portability |

| Authentication | JWT | Suitable for separate frontend/backend |

| Password security | Werkzeug hashing | Secure password storage |

| File storage | Cloudflare R2 | Scalable object storage |

| ML model | Random Forest | Suitable for tabular nonlinear data |

| ML preprocessing | OneHotEncoder and Pipeline | Consistent training and prediction |

| Production server | Gunicorn | Standard Python WSGI server |

| Deployment | Vercel and backend hosting | Independent frontend/backend deployment |

---

# 11. Honest Limitations to Mention

These are important if an interviewer asks what you would improve:

1. The model currently uses synthetic data, so real rental data would be needed for production-grade accuracy.

2. The reported model metrics are based on generated data.

3. The backend creates tables automatically with `db.create_all()`; a migration system such as Alembic would be better for production.

4. JWT tokens are stored in `localStorage`; secure HTTP-only cookies would reduce exposure to XSS attacks.

5. There are older ML engine files with a different feature schema. The active prediction path is `ml_loader.py`; those older engine classes should be consolidated or removed.

6. More backend validation could be added for negative rent, invalid area, invalid bedroom counts, and malformed requests.

7. Automated backend and frontend tests are not currently visible.

8. Database uniqueness constraints could be added for wishlist and review combinations.

9. Real-time communication between renters and customers is not included; the current implementation provides the owner’s email.

10. The system currently performs prediction when requested rather than continuously retraining the model.

---

# 12. Strong Project Explanation for Interviews

You can say:

> “I developed RentWise, a full-stack rental price fairness platform. The frontend is built with React, Vite, React Router, Axios, and Tailwind CSS. The backend is built with Flask and exposes REST APIs for authentication, property management, reviews, wishlists, image uploads, and rental analysis.

>

> I implemented two roles: renters can create and manage property listings, while customers can search, filter, save, review, and analyze properties. Authentication is handled using JWT, and passwords are hashed before being stored. SQLAlchemy is used as the ORM so the application can work with different relational databases. Images are stored in Cloudflare R2, while only their URLs are stored in the database.

>

> The main feature is the rental-price analysis system. I trained a Random Forest regression model using city, locality, bedrooms, and area as input features. When a customer analyzes a property, the backend predicts the expected rent and compares it with the listed rent. The system then classifies the property as overpriced, underpriced, or fairly priced.

>

> I chose this architecture because it separates the frontend, backend, database, object storage, and machine-learning components, making the project easier to maintain and deploy.”

---

# 13. Questions You Should Be Ready For

### Why did you use JWT?

Because the frontend and backend are separate applications. JWT provides stateless authentication for REST APIs.

### Why did you use SQLAlchemy?

It allows Python classes to represent database tables and makes database operations easier and more portable.

### Why Random Forest?

Because rental price depends on nonlinear relationships between location, area, and bedrooms. Random Forest performs well on structured tabular data.

### How is the price classified?

The actual rent is compared with the predicted rent. A difference greater than ₹2,000 is considered overpriced, a difference below -₹2,000 is underpriced, and everything else is fairly priced.

### How are passwords stored?

Only hashed passwords are stored using Werkzeug. Plain-text passwords are never saved.

### Why store images in R2?

Images are large binary files, so object storage is more scalable and appropriate than storing them directly in relational database rows.

### How do you prevent unauthorized editing?

The backend reads the user ID from the JWT and compares it with the property's `owner_id` before allowing updates or deletion.

### What happens if the ML model fails?

The analysis endpoint returns a failure response instead of crashing the application.

### What would you improve?

I would train the model on real rental data, add automated tests, use database migrations, strengthen validation, use HTTP-only cookies for authentication, and consolidate the duplicate ML engine implementations.

### What is the biggest limitation?

The current ML model is trained on synthetic data. It demonstrates the complete ML workflow, but real-world deployment would require verified rental-market data and continuous evaluation.

**Tasko — QueueCTL: Complete Project Description**

**1. Project Overview**

**QueueCTL** (codenamed Tasko) is a **CLI-based persistent background job queue** built entirely from the Python standard library. It simulates a production-grade distributed task system where:

- Jobs (arbitrary shell commands) are enqueued via a CLI
- One or more worker processes poll for and execute jobs concurrently
- Every job is tracked in durable, crash-safe persistent storage (SQLite)
- Failed jobs retry with configurable exponential backoff
- Jobs that exhaust their retry budget land in a Dead Letter Queue (DLQ)
- If a worker is killed mid-execution (even via SIGKILL / taskkill /F), the orphaned job is automatically detected and recovered by the next live worker
- Workers can be gracefully stopped from a separate terminal session
- An optional web dashboard provides a real-time UI over the same queue

The system is designed to demonstrate core backend engineering concepts — **concurrency control, crash recovery, state machines, atomic database operations, and process lifecycle management** — without hiding them behind framework abstractions.

---

**2. Tech Stack & Why Each Choice Was Made**

2.1 Python (3.11+)

**Why chosen:**

- Rapid development for a backend assignment — Python's expressiveness

allowed building a complete queue system with ~1,600 lines of application code

- First-class sqlite3 module built into the standard library (no driver installation)
- subprocess and threading modules give direct control over process spawning and concurrent worker loops
- signal module enables graceful SIGINT/SIGTERM shutdown handlers
- argparse produces professional CLI help output for free
- Easy to reason about in interviews — the code reads almost like pseudocode

**Why not other languages:**

| Alternative | Why Rejected |
|---|---|
| **Node.js** | Excellent for async I/O and web APIs, but this project is *not* a web app — it's a process manager with subprocess execution, database locking, and signal handling. Node's event loop model is less natural for blocking subprocess management and SQLite write serialization. |
| **Go** | Great concurrency primitives, but the assignment environment was Windows + Python, and Go's SQLite bindings require CGo (cross-compilation complexity). Python was simpler for the scope. |
| **Java / C#** | Too much ceremony (build systems, class boilerplate) for a focused assignment. The queue logic would be buried under framework scaffolding. |

2.2 SQLite with WAL Mode

**Why chosen:**

- **Zero external services** — no Redis server, no RabbitMQ broker, no PostgreSQL daemon to install. The entire system runs from a

single jobs.db file.

- **ACID transactions** — SQLite provides full transaction support with BEGIN IMMEDIATE for writer serialization, which is the foundation of the atomic job claiming mechanism.
- **WAL (Write-Ahead Logging) mode** — enables concurrent readers while a writer is active, critical for the dashboard reading queue state while workers write.
- **Cross-process safety** — SQLite's file-level locking ensures that multiple worker processes (each in a separate terminal) cannot corrupt the database.
- **Persistence across restarts** — jobs survive process crashes, machine reboots, and full system restarts because they're written to disk, not held in memory.
- **busy_timeout** — configured at 5,000ms so competing writers block briefly instead of failing immediately. This is important when multiple workers are racing to claim jobs.

**Specific SQLite features exploited:**

- BEGIN IMMEDIATE — acquires the write lock *before* reading, preventing TOCTOU races
- UPDATE ... RETURNING * — atomically selects and updates a row in a single statement
- isolation_level=None — disables Python's auto-transaction so the code controls transaction boundaries explicitly
- PRAGMA foreign_keys=ON — enforces referential integrity
- PRAGMA journal_mode=WAL — enables concurrent reads during writes
- Row factory — enables column access by name (row["state"]) for readable code

**Why not other databases / storage:**

| Alternative | Why Rejected |
| --- | --- |
| **Redis** | Redis is an in-memory store. The assignment specifically requires *persistence* and correctness under process crashes. While Redis has RDB/AOF persistence, it's primarily designed for caching and pub/sub, not durable queue semantics with transaction isolation. Also requires running a separate server process. |
| **RabbitMQ / Kafka** | Message brokers are the standard production choice for job queues, but they're infrastructure-heavy. The assignment tests queue *design* (claiming, recovery, backoff), not infrastructure wiring. A broker would abstract away exactly the logic being evaluated. |
| **PostgreSQL** | Powerful and production-ready, but requires external setup (install, configure, start a daemon, create a database). SQLite demonstrates the same transaction safety concepts without any ops overhead. The claim query uses the same UPDATE ... WHERE id = (SELECT ...) pattern that would work in PostgreSQL. |
| **JSON files + file locks** | Fragile across platforms. File locking semantics differ between Windows and POSIX. No transaction rollback, no concurrent reader safety, no indexing. |
| **In-memory queues** (queue.Queue) | Fails the persistence requirement entirely. Jobs would be lost on any crash or restart. |

2.3 Standard Library Only (Zero Runtime Dependencies)

The application has **zero pip-installable runtime dependencies**. Every import comes from the Python standard library:

| Module | Purpose |
|---|---|
| argparse | CLI parser with subcommands, help text, and input validation |
| sqlite3 | Database access, transactions, WAL mode |
| threading | Concurrent worker loops within a single process |
| subprocess | Shell command execution for jobs |
| signal | Graceful SIGINT/SIGTERM shutdown handling |
| json | Job payload serialization, list --json output |
| os / shutil / socket | Process IDs, shell detection, hostname for worker identification |
| uuid | Unique worker ID generation |
| dataclasses | Immutable domain objects (Job, CommandResult, WorkerSettings) |
| enum | Type-safe state transition actions |
| http.server | Optional dashboard (bonus) with no Flask/FastAPI dependency |
| importlib.resources | Load schema.sql from the package at runtime |
| functools.lru_cache | Singleton database connection per process |
| ctypes | Platform-specific orphan-process prevention (Windows Job Objects, Linux prctl) |

**Why not use frameworks:**

| Alternative | Why Rejected |
|---|---|
| **FastAPI / Flask** | This is not a web API project. The CLI is the primary interface. Adding a web framework would pull in uvicorn, starlette, pydantic, etc. — 20+ transitive dependencies to serve one optional bonus dashboard. The |

| Alternative | Why Rejected |
|---|---|
| | stdlib http.server handles it in 100 lines. |
| **Celery** | Celery *is* a distributed task queue, which is exactly what this project builds from scratch. Using Celery would defeat the purpose — it's like using sorted() in a sorting algorithm assignment. |
| **SQLAlchemy** | An ORM hides the exact SQL statements that make atomic claiming work. The assignment requires pointing reviewers to specific lines of SQL. Raw sqlite3 makes BEGIN IMMEDIATE and UPDATE ... RETURNING visible and auditable. |
| **Pydantic** | The Job model has 11 fields. A frozen @dataclass with a manual to_dict() method is 30 lines. Adding Pydantic for validation would add a dependency for marginal benefit at this model count. |
| **Click / Typer** | argparse is built-in and provides subcommands, type coercion, choices, and --help for free. Click/Typer are nicer but add a dependency for cosmetic improvement. |

2.4 GitHub Actions CI/CD

**Why added:**

- Automated testing on every push to main ensures the 26-test suite passes in a clean environment (not just on the developer's machine)
- Docker image build verification ensures the Dockerfile stays valid as code evolves
- Demonstrates CI/CD as a professional engineering practice

**What it does:**

1. Checks out the repository
2. Sets up Python 3.12
3. Installs dependencies from requirements.txt + pytest

4. Runs the full pytest suite (26 tests)
5. Builds a Docker image tagged with the commit SHA

2.5 Docker

**Why added:**

- Containerizes the CLI tool for reproducible deployment
- Uses python:3.12-slim as the base image to keep the image small (~150MB vs ~900MB for the full Python image)
- Two-stage dependency install: requirements.txt first (cached layer), then application code — rebuilds are fast when only code changes

**Why not Kubernetes / Docker Compose / heavy infra:**

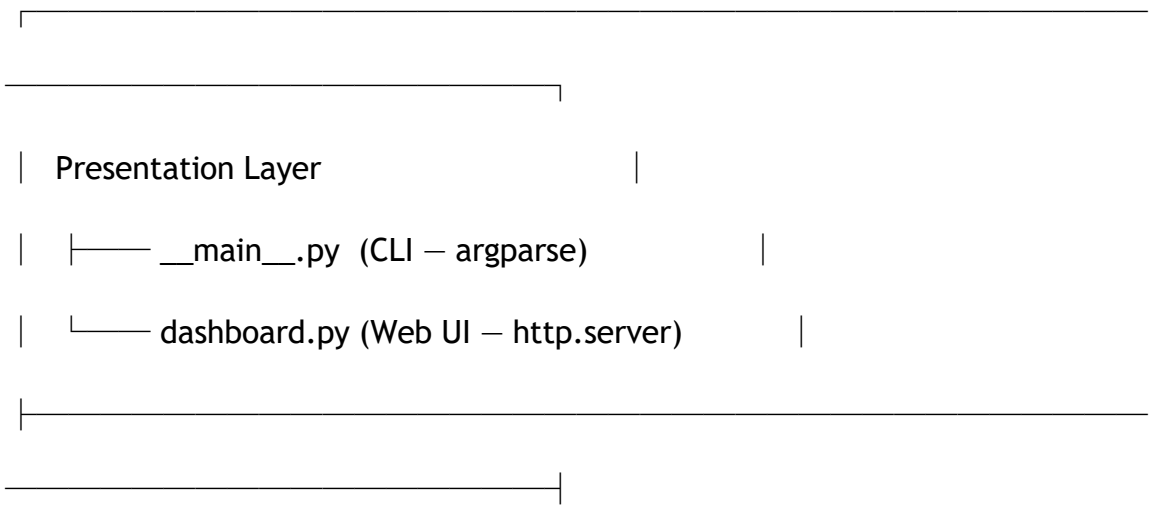
- The assignment evaluates queue *mechanics*, not deployment orchestration
- The Dockerfile proves the app runs in an isolated Linux environment, which is sufficient for the scope

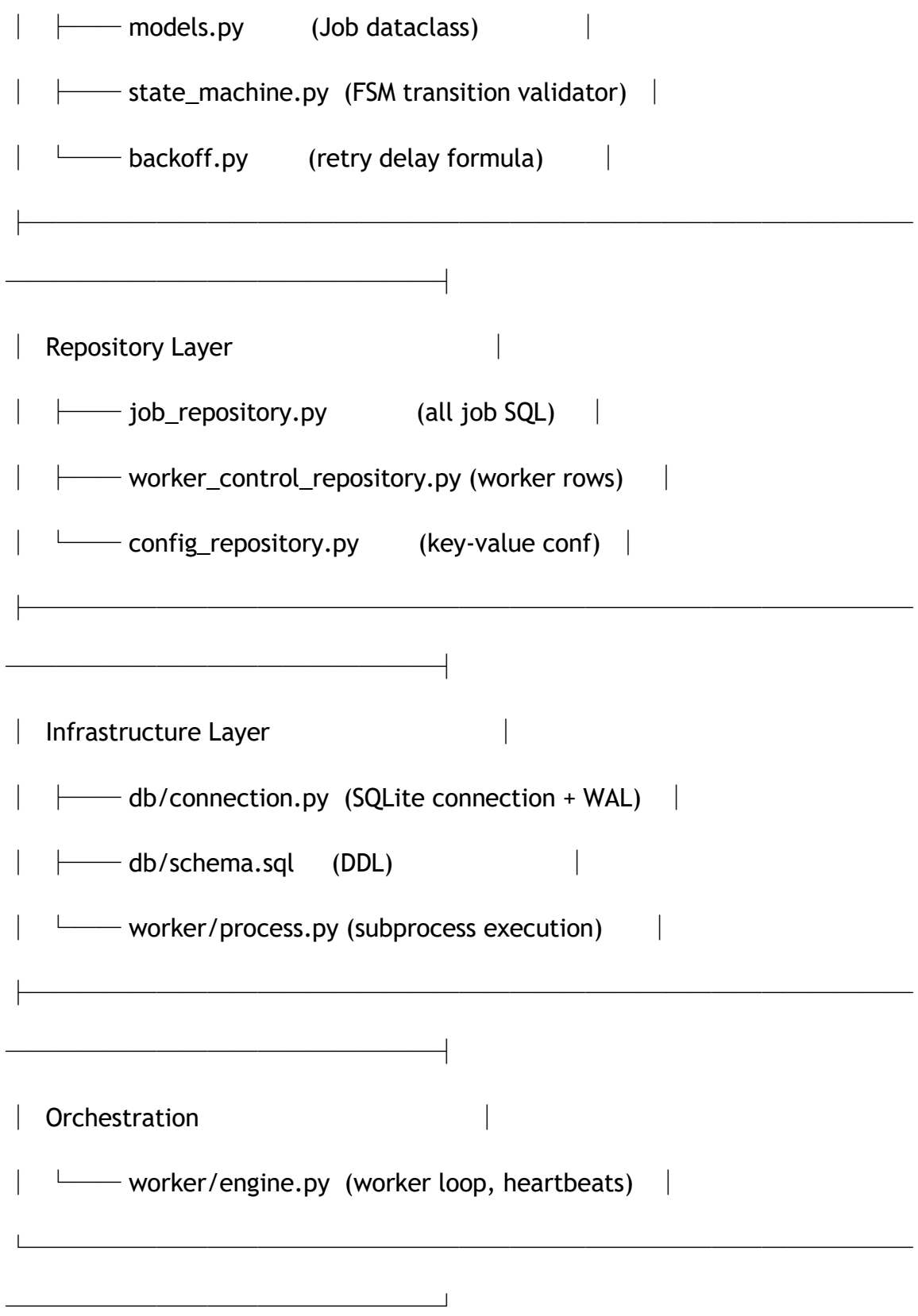
---

**3. Architecture & Design Decisions**

3.1 Layered Architecture

The codebase follows a **three-layer architecture** with clear separation of concerns:






**Why this architecture:**

- Each layer can be tested independently (unit tests for domain, integration tests for repositories, E2E tests for CLI)
- The repository pattern isolates *all* SQL in one place — reviewers can find every transaction boundary by reading job_repository.py alone

- The domain layer has zero database imports — state transitions are pure logic
- The dashboard reuses the same repository layer as the CLI, so it doesn't introduce a second persistence or locking model

**Rejected alternative: Active Record pattern** (Job.save(), Job.claim()) — this spreads transaction boundaries across callers, making it harder to audit atomicity and harder to test.

3.2 Database Schema

Three tables, no more:

sql

```
-- The job queue itself
CREATE TABLE jobs (
    id          TEXT PRIMARY KEY,
    command     TEXT NOT NULL,
    state       TEXT NOT NULL CHECK (state IN
('pending','processing','completed','failed','dead')),
    attempts    INTEGER NOT NULL DEFAULT 0,
    max_retries INTEGER NOT NULL,
    run_at      TEXT NOT NULL,        -- backoff scheduling
    worker_id   TEXT,                 -- which worker owns this job
    last_error  TEXT,                 -- stderr/error from last failure
    exit_code   INTEGER,
    created_at  TEXT NOT NULL,
    updated_at  TEXT NOT NULL         -- heartbeat freshness for crash detection
);
-- Performance index for the claim query (avoids full table scan)
CREATE INDEX idx_jobs_claim ON jobs (state, run_at, created_at);
-- Runtime configuration (persisted across restarts)
```

```
CREATE TABLE config (
    key   TEXT PRIMARY KEY,
    value TEXT NOT NULL
);
-- Worker registration and cross-terminal stop signaling
CREATE TABLE worker_control (
    worker_id      TEXT PRIMARY KEY,
    pid            INTEGER NOT NULL,
    hostname       TEXT NOT NULL,
    stop_requested INTEGER NOT NULL DEFAULT 0,
    last_heartbeat TEXT NOT NULL,
    registered_at  TEXT NOT NULL
);
CREATE INDEX idx_worker_control_heartbeat ON worker_control
(last_heartbeat);
```

**Why only three tables:**

- jobs holds the queue. The CHECK constraint enforces valid state values at the database level.
- config stores tunable parameters (max-retries, backoff-base, poll-interval, stale-threshold, heartbeat-interval). Persisting config in SQLite instead of a YAML file avoids file-lock races between config writes and job claims.
- worker_control enables cross-terminal worker shutdown and crash detection. No job_events audit table — the assignment doesn't require historical tracking, so the schema stays minimal.

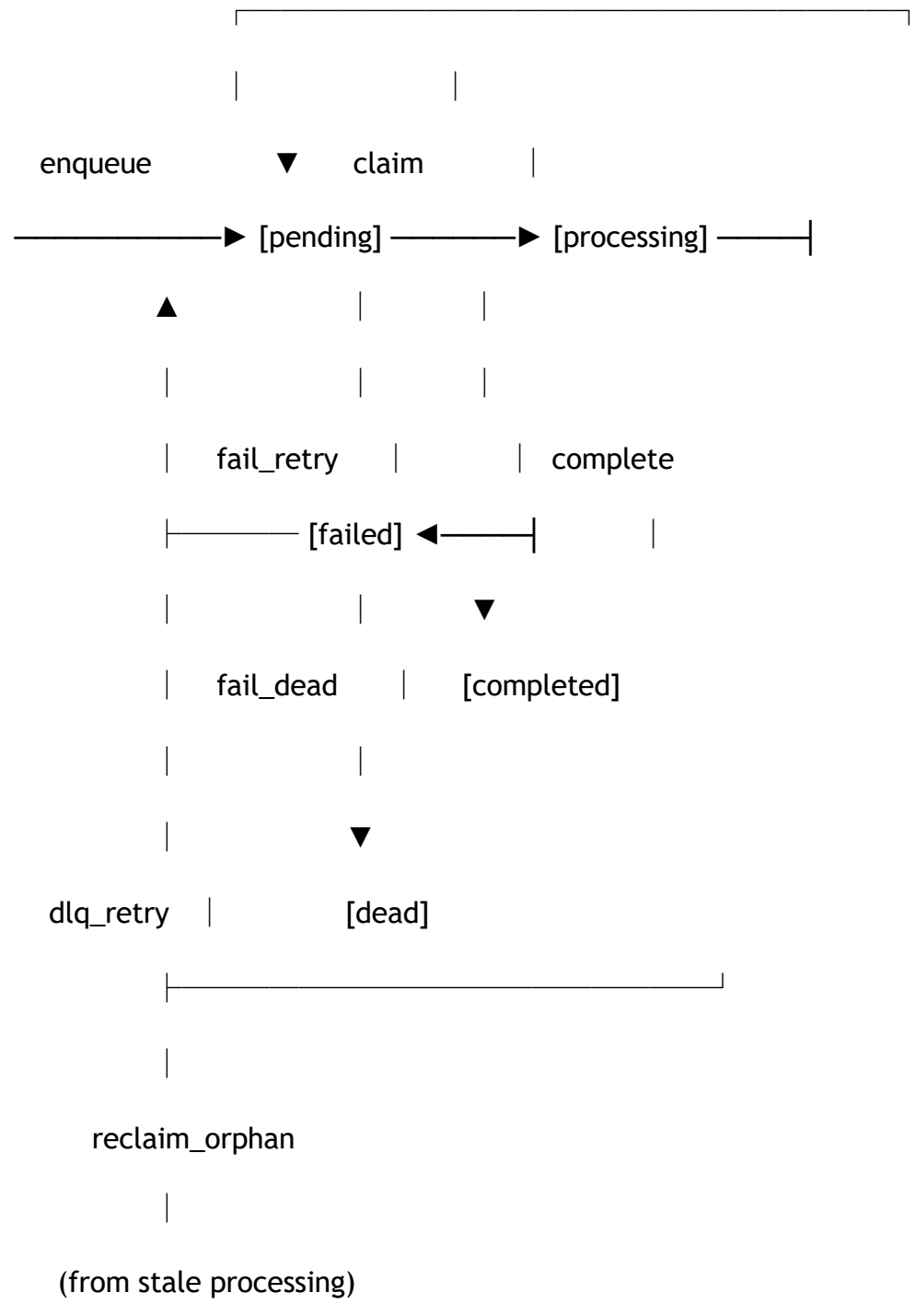
**Why TEXT for timestamps instead of INTEGER (Unix epoch):**

- ISO-8601 strings are human-readable in direct database queries (SELECT * FROM jobs)

- SQLite's datetime() function works natively with ISO strings for stale-job comparisons
- Trade-off: slightly more storage, but negligible for this scale

3.3 Job State Machine (Finite State Machine)

Jobs follow a strict lifecycle enforced by an application-level FSM:




**Seven allowed transitions, every other transition is rejected:**

| From | Action | To |
|---|---|---|

| From | Action | To |
|---|---|---|
| pending | CLAIM | processing |
| failed | CLAIM | processing |
| processing | COMPLETE | completed |
| processing | FAIL_RETRY | failed |
| processing | FAIL_DEAD | dead |
| processing | RECLAIM_ORPHAN | pending |
| dead | DLQ_RETRY | pending |

**Why an application-level FSM instead of only SQL CHECK constraints:**

- SQL CHECK enforces valid *values* (state IN ('pending', 'processing', ...)) but cannot enforce valid *transitions* (e.g., it can't prevent completed → pending)
- Centralizing transitions in state_machine.py makes the FSM auditable, testable, and visible in code review
- Every repository method calls StateTransitionValidator.target_state() before writing to the database

**Rejected alternative:** Scattered if/else checks in the worker loop — error-prone, not testable in isolation, and would get worse as commands grow.

3.4 Atomic Job Claiming (The Core Concurrency Problem)

The single most critical design decision: **how do two workers avoid claiming the same job?**

python

```
# job_repository.py — claim_next_job()
self._conn.execute("BEGIN IMMEDIATE")
row = self._conn.execute("""
    UPDATE jobs
```

```
    SET state = 'processing',
        worker_id = ?,
        attempts = attempts + 1,
        updated_at = ?
    WHERE id = (
        SELECT id FROM jobs
        WHERE state IN ('pending', 'failed')
          AND datetime(run_at) <= datetime(?)
        ORDER BY created_at ASC
        LIMIT 1
    )
    RETURNING *
""", (worker_id, now, now)).fetchone()
self._conn.execute("COMMIT")
```

**How it works:**

1. BEGIN IMMEDIATE acquires SQLite's **RESERVED write lock** before any read. Only one process can hold this lock at a time.
2. While the lock is held, a single UPDATE ... WHERE id = (SELECT ...) RETURNING * atomically finds the oldest eligible job and transitions it to processing in one statement.
3. A competing worker blocks on BEGIN IMMEDIATE until the first commits, then sees the job already in processing and cannot claim it.

**Why BEGIN IMMEDIATE instead of BEGIN (deferred):**

- BEGIN (deferred) acquires locks lazily. Two workers could both start deferred transactions, both SELECT the same pending job, and then one would fail on COMMIT with a SQLITE_BUSY error. BEGIN IMMEDIATE prevents this by serializing at the start.

**Why a single UPDATE ... WHERE id = (SELECT ...) instead of**

**separate SELECT then UPDATE:**

- A separate SELECT followed by UPDATE has a time-of-check/time-of-use (TOCTOU) race unless carefully wrapped. The combined statement eliminates the window entirely.

**Why ORDER BY created_at ASC (FIFO):**

- FIFO ordering is fair and predictable. The idx_jobs_claim index on (state, run_at, created_at) supports this query without a full table scan, so claiming stays O(log n) as the queue grows.

**The concurrency test proves this works:**

- test_concurrent_claims_only_one_winner spawns **10 threads** (each with its own SQLite connection, simulating separate OS processes) racing to claim a single job. Exactly one wins; the other nine get None.

3.5 Crash Recovery (SIGKILL Handling)

**The problem:** When a worker is killed with SIGKILL (or taskkill /F on Windows), Python's atexit handlers, try/finally blocks, and signal handlers **do not run**. The job remains in processing state with a stale updated_at timestamp.

**The solution — dual-guard stale detection:**

Every worker loop calls reclaim_stale_jobs() before trying to claim new work. This method:

1. Finds all processing jobs whose updated_at is older than stale_threshold seconds
2. For each stale job, checks whether the owning worker's last_heartbeat is **also** stale or missing
3. Only reclaims the job if **both** the job timestamp and the worker heartbeat are stale

**Why the dual guard:**

- A single guard on updated_at alone would incorrectly reclaim jobs from a live worker running a legitimately long command (e.g., a 60-second build). The worker may be alive and heartbeating, but the

job's updated_at could lag between heartbeat intervals.

- The dual guard ensures: "this job is stale AND the worker that owns it is also dead/unresponsive." Only then is it safe to reclaim.

**Recovery timing with default settings:**

- stale-threshold = 30 seconds
- poll-interval = 2 seconds
- Worst-case recovery: ~32 seconds (30s staleness + one 2s poll cycle)
- This is well under the assignment's 60-second requirement

**Worker heartbeat system:**

- While executing a job, the worker
  calls heartbeat() every heartbeat_interval seconds (default: 5)
- This updates **two** things: worker_control.last_heartbeat (worker liveness) and jobs.updated_at (job freshness)
- The heartbeat is implemented
  via process.communicate(timeout=heartbeat_interval) — the worker tries to read subprocess output, and on timeout (subprocess still running), it heartbeats and retries

**Orphan child process prevention (best-effort, not relied upon for correctness):**

- **Linux:** os.setsid() creates a new process group,
  and prctl(PR_SET_PDEATHSIG, SIGKILL) asks the kernel to kill the child if the parent dies
- **Windows:** A Windows Job Object
  with JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE flag is created and the child is assigned to it — when the job handle closes (worker dies), the child is terminated
- These are defense-in-depth measures. The actual correctness guarantee comes from the database stale-job mechanism.

3.6 Cross-Terminal Worker Shutdown

**The problem:** The user starts workers in Terminal A (queuectl worker start --count 3) and wants to stop them from Terminal B (queuectl worker stop). How do the two terminals communicate?

**The solution — database-based stop signaling:**

1. When workers start, each registers a row in worker_control with its worker_id, pid, hostname, and last_heartbeat
2. queuectl worker stop sets stop_requested = 1 for all recently active workers
3. Each worker checks stop_requested on every heartbeat cycle. If stop is requested, it finishes its current in-flight job (no new claims) and exits cleanly

**Why database rows instead of alternatives:**

| Alternative | Why Rejected |
|---|---|
| **PID files** | Fragile with --count 3 (three workers). PID files can be overwritten, become stale on crashes, and suffer from PID reuse. |
| **POSIX signals** (SIGTERM) | Not portable — signal semantics differ between Windows and Unix. Targeting the correct PID from a separate terminal is also fragile. |
| **TCP control socket** | Workable, but more moving parts than needed. Port conflicts, connection handling, and error recovery add complexity. |
| **Named pipes / Unix sockets** | Platform-specific. Windows named pipes and Unix domain sockets have different APIs. |

The database approach is simple, uses the same storage that already coordinates the queue, and works identically across Windows and Linux.

3.7 Exponential Backoff

When a job fails but has retries remaining, it's moved to failed state with a

future run_at timestamp:

python

delay = base ** attempts  # e.g., 2^1=2s, 2^2=4s, 2^3=8s

run_at = now + delay

- base is configurable via queuectl config set backoff-base <N> (default: 2)
- attempts is the current attempt count (incremented at claim time)
- The claim query skips jobs where datetime(run_at) > datetime(now), so backoff is enforced by the queue itself

**Why a standalone function instead of a BackoffPolicy class:**

- One formula (base^attempts) with no pluggable strategies. A class would add ceremony without benefit. Jitter or custom policies can extend the function later without breaking callers.

**Why base^attempts instead of base * 2^attempts:**

- Simpler formula, configurable base covers the same range. With base=2: delays are 2s, 4s, 8s, 16s, ... which is standard exponential backoff.

3.8 Dead Letter Queue (DLQ)

When a job exhausts its retry budget (attempts >= max_retries), it moves to dead state — the Dead Letter Queue.

**dlq retry <id> resets attempts to 0.** This is a deliberate design decision:

- A DLQ retry is an *explicit operator decision* to give the job a fresh execution budget after investigating the root cause
- Keeping the old attempt count would immediately send many jobs back to dead, making the retry command useless
- Trade-off: the cumulative attempt count is lost. A production extension could add a job_events audit table without changing the core state machine.

3.9 Configuration Semantics

Five tunable parameters, each with specific application timing:

| Key | Default | When Applied | Implication |
|---|---|---|---|
| max-retries | 3 | At enqueue time (copied to job) | Changing it affects *future* jobs only |
| backoff-base | 2 | At failure time (read when scheduling retry) | Changing it affects future retries of *existing* jobs |
| poll-interval | 2s | At worker start | Restart workers after changing |
| stale-threshold | 30s | At worker start | Restart workers after changing |
| heartbeat-interval | 5s | At worker start | Restart workers after changing |

**Why persist config in SQLite instead of a YAML/TOML file:**

- Same database already coordinates workers across processes. One store means one backup story and no file-lock race between config writes and job claims.
- The config set command provides a persisted CLI interface — no need to edit files.

3.10 Shell Execution (Platform Portability)

Jobs are shell commands (echo hello, sleep 2, exit 1). The process runner detects the platform:

- **Windows:** Defaults to PowerShell (powershell.exe -NoProfile -ExecutionPolicy Bypass -Command ...) because PowerShell supports sleep, echo, and exit 1 — the standard assignment examples. Falls back to cmd.exe /C. The QUEUECTL_SHELL environment variable overrides the shell.
- **Linux/macOS:** Uses /bin/sh -c ... (standard POSIX shell execution)

**Why PowerShell as default on Windows instead of cmd.exe:**

- cmd.exe doesn't have a native sleep command
- PowerShell handles all assignment example commands (sleep N, echo text, exit N) reliably

3.11 Optional Web Dashboard (Bonus)

A single-page web dashboard built with **zero frontend dependencies** — no React, no npm, no build step:

- Backend:
  Python http.server.ThreadingHTTPServer with BaseHTTPRequestHandler
- Frontend: Inline HTML/CSS/JavaScript embedded as a Python string constant in dashboard.py
- API endpoints: GET /api/status, GET /api/jobs?state=X, POST /api/enqueue, POST /api/dlq/retry, POST /api/worker/stop
- Auto-refreshes every 3 seconds via setInterval
- Responsive CSS grid layout

**Key design constraint:** The dashboard is a thin layer over the *same* repository classes the CLI uses. It adds no new persistence path, no new locking model, and no runtime dependency. It can be removed entirely without affecting the core system.

3.12 Worker ID Generation

python

worker_id = f"{hostname}-{pid}-{index}-{uuid4().hex[:8]}"

# Example: DESKTOP-ABC-12345-1-a3f7b2c1

Contains hostname (machine identification), PID (process identification), index (thread number within --count N), and a random suffix (uniqueness across restarts with PID reuse).

3.13 Connection Management

python

@lru_cache(maxsize=1)

```
def get_connection(db_path=None):
    return open_connection(db_path)
```

One cached connection per process. SQLite recommends one writer connection per process for writer-heavy workloads. The lru_cache prevents accidental creation of multiple connections from different repository instances within the same process.

Workers spawned via --count 3 run in separate threads but share a process — each thread opens its **own** connection via open_connection() (not the cached singleton) to avoid SQLite's thread-safety limitations.

**Rejected alternative:** Connection pool — overkill for a single-file SQLite CLI queue.

---

**4. What the Project Actually Does (Feature by Feature)**

4.1 Job Enqueueing

bash

```
queuectl enqueue '{"id":"job1","command":"sleep 2"}'
```

- Validates the JSON payload (requires id and command as non-empty strings)
- Copies max-retries from global config (or uses per-job override)
- Sets state = 'pending', run_at = now, attempts = 0
- Returns the job as JSON to stdout

4.2 Worker Management

bash

```
queuectl worker start --count 3   # Start 3 concurrent worker loops
queuectl worker stop              # Graceful stop from another terminal
```

- --count 1: runs directly in the main thread (simpler stack traces)
- --count N > 1: spawns N threads, each with its own DB connection and worker loop

- SIGINT/SIGTERM installs a shutdown handler that finishes the current job before exiting
- worker stop sets stop_requested = 1 in the database for all active workers

4.3 Queue Inspection

bash

```
queuectl status                  # Counts per state + active workers
queuectl list --state pending      # Human-readable table
queuectl list --state pending --json  # Machine-readable JSON array
```

- list --json writes **only** a JSON array to stdout (no headers, no logs) — suitable for piping to jq or other tools. All worker diagnostics go to stderr.

4.4 Dead Letter Queue

bash

```
queuectl dlq list --json          # List dead jobs
queuectl dlq retry job1           # Re-enqueue with fresh attempts
```

4.5 Configuration

bash

```
queuectl config set max-retries 5
queuectl config set backoff-base 3
queuectl config list
```

- Validates input ranges (e.g., backoff-base >= 1, stale-threshold between 5-59)

4.6 Dashboard (Bonus)

bash

```
queuectl dashboard --host 127.0.0.1 --port 8000
```

- Real-time metric cards: pending, processing, completed, failed, dead, active workers
- Tab-based job listing with retry buttons for DLQ jobs
- Enqueue form

- Worker stop button

---

**5. Testing Strategy**

5.1 Test Structure

**26 tests** across 5 test files, organized by scope:

| File | Scope | Tests | What It Proves |
|---|---|---|---|
| test_state_machine.py | Unit | 3 | FSM allows valid transitions, rejects invalid ones (e.g., completed → processing) |
| test_backoff.py | Unit | 3 | Backoff formula produces correct delays, rejects invalid inputs |
| test_repository.py | Integration | 10 | Enqueue, claim, FIFO order, concurrent claim (10 threads), mark complete/failed, DLQ retry, stale reclaim (dual guard), config seeding |
| test_cli_worker.py | E2E | 5 | Full CLI round-trips through real subprocesses — basic completion, retry→DLQ, 12 jobs across 3 workers, SIGKILL recovery, persistence across restart |
| test_dashboard.py | Integration | 1 | Dashboard HTML, |

| File | Scope | Tests | What It Proves |
|---|---|---|---|
| | | | enqueue API, status API, job listing API, worker stop API |

5.2 Test Isolation

Every test gets a **fresh temporary SQLite database** via the temp_db and cli_env fixtures:

python

```
@pytest.fixture()
def temp_db(tmp_path, monkeypatch):
    db_path = tmp_path / "test.db"
    monkeypatch.setenv("QUEUECTL_DB_PATH", str(db_path))
    reset_connection_cache()
    conn = get_connection(db_path)
    initialize_database(conn)
    yield conn
    conn.close()
    reset_connection_cache()
```

5.3 Key Tests That Prove Correctness

**Atomic claiming (10 concurrent threads):**

python

```
def test_concurrent_claims_only_one_winner():
    # 10 threads, each with own DB connection, race to claim 1 job
    # Exactly 1 gets the job, other 9 get None
    assert results.count("only-one") == 1
    assert sum(1 for item in results if item is not None) == 1
```

**SIGKILL crash recovery:**

python

```
def test_sigkill_recovery_completes_without_duplicate_execution():
    # 1. Start worker, let it claim job
    # 2. Kill worker with .kill() (SIGKILL equivalent)
    # 3. Start new worker
    # 4. Job reaches 'completed', execution file has exactly one line
    assert lines == ["crash-1"]  # executed exactly once
```

**12 jobs across 3 workers, each executes exactly once:**

python

```
def test_many_jobs_across_workers_execute_once():
    # Each job appends its ID to a shared file
    assert sorted(lines) == sorted(job_ids)     # all completed
    assert len(lines) == len(set(lines))        # no duplicates
```

---

**6. Project File Structure**

```
Tasko/
├── .github/
│   └── workflows/
│       └── ci.yml                # GitHub Actions CI pipeline
├── queuectl/
│   ├── __init__.py
│   ├── __main__.py               # CLI entry point (argparse)
│   ├── dashboard.py              # Optional web dashboard (bonus)
│   ├── db/
│   │   ├── __init__.py
│   │   ├── connection.py         # SQLite connection + WAL + schema
```

```
bootstrap
│   │   └── schema.sql            # DDL for jobs, config, worker_control
│   ├── domain/
│   │   ├── __init__.py
│   │   ├── backoff.py            # Exponential backoff formula
│   │   ├── models.py             # Job dataclass + serialization
│   │   └── state_machine.py      # FSM transition validator
│   ├── repository/
│   │   ├── __init__.py
│   │   ├── config_repository.py  # Persisted key-value config
│   │   ├── job_repository.py     # All job SQL + atomic claiming
│   │   └── worker_control_repository.py  # Worker registration + stop
signaling
│   └── worker/
│       ├── __init__.py
│       ├── engine.py             # Worker loop, heartbeats, recovery
│       └── process.py            # Platform-aware subprocess execution
├── tests/
│   ├── conftest.py               # Shared fixtures (temp_db, cli_env)
│   ├── test_backoff.py           # Unit: backoff formula
│   ├── test_cli_worker.py        # E2E: full CLI round-trips
│   ├── test_dashboard.py         # Integration: dashboard API
│   ├── test_repository.py        # Integration: repository operations
│   └── test_state_machine.py     # Unit: FSM transitions
├── Dockerfile                    # Container image (python:3.12-slim)
```

```
├── .dockerignore              # Exclude .venv, .git, tests from image
├── .gitignore
├── pyproject.toml             # Package metadata + pytest config
├── requirements.txt           # Dev dependency: pytest>=8.0.0
├── DECISIONS.md               # Assignment interview defense answers
├── DEMO.md                    # Demo recording script
└── README.md                  # Setup, usage, architecture overview
```

---

**7. Summary of All Major Decisions**

| Decision | Choice | Why | Rejected Alternative |
|---|---|---|---|
| Language | Python 3.11+ | Fast dev, stdlib sqlite3/subprocess /signal | Node.js, Go, Java |
| Database | SQLite + WAL | Zero setup, ACID, cross-process safety | Redis, RabbitMQ, PostgreSQL |
| Dependencies | Zero runtime deps | Portable, explainable, focused | FastAPI, Celery, SQLAlchemy |
| Claiming | BEGIN IMMEDIATE + single UPDATE...RETURNING | Atomic, no TOCTOU race | Separate SELECT→UPDATE, advisory locks |
| Crash recovery | Dual-guard stale detection (job + | Avoids reclaiming live long-running | Single timestamp |

| Decision | Choice | Why | Rejected Alternative |
| --- | --- | --- | --- |
| | worker heartbeat) | jobs | check, PID probing |
| Worker stop | Database stop_requested flag | Cross-platform, cross-terminal | PID files, POSIX signals, TCP socket |
| State transitions | Application-level FSM | Auditable, testable, prevents invalid transitions | SQL CHECK only, scattered if/else |
| Backoff | base^attempts function | Simple, configurable, extensible | BackoffPolicy class hierarchy |
| DLQ retry | Reset attempts to 0 | Operator intent: fresh budget after investigation | Keep cumulative count |
| Config storage | SQLite config table | Same store as queue, no file-lock races | YAML file, environment variables only |
| Shell execution | Platform-detected (PowerShell / /bin/sh) | Assignment examples work on both OS | cmd.exe only, hardcoded /bin/bash |
| Dashboard | stdlib http.server, inline HTML | Zero deps, same repository layer | Flask, FastAPI, |

| Decision | Choice | Why | Rejected Alternative |
| --- | --- | --- | --- |
| | | | React SPA |
| Connection | lru_cache singleton per process | SQLite single-writer recommendation | Connection pool, global variable |
| CI/CD | GitHub Actions (pytest + Docker build) | Automated verification on every push | Jenkins, GitLab CI, no CI |
| Containerization | python:3.12-slim Dockerfile | Reproducible, small image | Full python image, Alpine, no Docker |

Volt Path

I'll trace the project from its build files and the main frontend/backend entry points, then summarize the architecture, technologies, and the rationale behind each choice in presentation-ready language.

The project is a two-part application: a Vite/React client and a Spring Boot API with JPA persistence. I'm checking the routing logic, API endpoints, database model, and planner UI now so the explanation reflects what the code actually implements rather than just the dependency list.

The core behavior is clear: the backend calculates an EV route using vehicle range, a 10% reserve, connector-compatible charging stations, geographic distance, and a greedy "best next station" strategy; the frontend lets users enter or select coordinates, submits the trip, and visualizes the returned stops. I'm doing one final pass over the UI components and database/test setup to

explain the user workflow and the technology tradeoffs accurately.

## Project Overview

**VoltPath** is an electric vehicle trip-planning application. It calculates whether an EV can travel from an origin to a destination and recommends charging stations when the vehicle cannot complete the journey on one charge.

The system includes:

- Vehicle selection
- Origin and destination coordinate input
- Interactive map selection
- Charging station visualization
- EV range-aware route calculation
- Connector-compatible charging stops
- Charging station availability management
- Reachable or unreachable route results

## Technologies Used

### Frontend

- **React**: Builds the user interface using reusable components such as `TripForm`, `MapView`, and `RouteResult`.
- **Vite**: Provides fast frontend development and production builds.
- **React Router**: Handles navigation between the landing page and planner page.

- **React Leaflet and Leaflet**: Displays the interactive map, charging stations, markers, popups, and routes.
- **Three.js**: Creates the animated “hyperspeed” visual effect on the landing page.
- **CSS**: Provides the custom layout, glass-style panels, dark map design, and responsive styling.

Relevant files: `App.jsx`, `PlannerPage.jsx`, `MapView.jsx`, `TripForm.jsx`

### Backend

- **Java 17**: Provides a strongly typed and reliable language for the backend.
- **Spring Boot**: Creates the REST API and manages the application structure.
- **Spring Web**: Provides REST controllers such as `/api/routes/calculate`.
- **Spring Data JPA**: Connects Java entities to database tables and simplifies database operations.
- **Spring Validation**: Validates incoming route requests.
- **Spring Security**: Configures CORS, stateless requests, and API security behavior.
- **Lombok**: Reduces repetitive getter, setter, and constructor code.
- **Maven**: Manages dependencies, compilation, testing, and packaging.

Relevant files: `pom.xml`, `VoltPathApplication.java`, `RoutingController.java`

### Database

- **PostgreSQL**: Used as the main relational database.

- **H2**: Used for lightweight development or testing.
- **JPA entities**: Represent vehicles, charging stations, routes, users, and route stops.
- **SQL seed data**: Provides initial vehicles and charging stations.

Relevant files: `application.yml`, `application-h2.yml`, `schema.sql`

## How the Routing Works

The main logic is implemented in `RoutingService.java`.

1. The selected vehicle is retrieved from the database.
2. Its maximum range is read.
3. A 10% safety buffer is applied to avoid planning routes using the complete theoretical range.
4. The Haversine formula calculates the distance between coordinates.
5. If the destination is within the effective range, the route has no charging stops.
6. Otherwise, the system searches for compatible charging stations.
7. Stations must:
   - Be within the vehicle’s remaining range
   - Support the correct connector type
   - Move the route closer to the destination
8. The closest useful station is selected.
9. The process repeats until the destination is reachable or no suitable station exists.
10. The response includes distance, route status, vehicle information, and charging stops.

This is a **greedy routing algorithm**. It is simpler and faster than a complex global optimization algorithm, making it suitable for a prototype or academic project.

## Why These Technologies Were Chosen

### Why React instead of Angular or plain JavaScript?

React is component-based, so the interface can be divided into reusable parts such as the form, map, and result panel. It is lighter and simpler to start with than Angular, while being more maintainable than writing the entire interface with plain JavaScript.

### Why Vite instead of Create React App?

Vite provides faster startup, faster hot reload, and a smaller modern configuration. It is well suited for a separate React frontend.

### Why Leaflet instead of Google Maps?

Leaflet is open source, lightweight, and does not require a paid Google Maps API key for basic map rendering. It works well for displaying charging stations and route markers.

### Why Three.js?

Three.js was used only for the animated visual experience on the landing page.

It creates a more engaging EV and futuristic transport theme without affecting the core routing functionality.

### Why Spring Boot instead of Node.js or Django?

Spring Boot provides:

- Strong Java typing
- Clear controller-service-repository architecture
- Excellent database integration through JPA
- Built-in validation and security support
- Easy REST API development

Node.js could also have been used, but Spring Boot is a strong choice for structured enterprise-style backend applications.

### Why PostgreSQL instead of MongoDB?

The project contains structured relationships:

- Users own vehicles
- Routes use vehicles
- Routes contain route stops
- Route stops reference charging stations

A relational database is therefore more suitable than MongoDB. PostgreSQL also provides reliable constraints, indexes, and numeric data types for coordinates and vehicle information.

### Why Haversine distance?

The Haversine formula calculates the approximate distance between two points on Earth using latitude and longitude. It is simple, fast, and appropriate for calculating straight-line geographic distance in this project.

### Why H2?

H2 allows the application to run without requiring a full PostgreSQL setup during testing or local development. It provides an in-memory database that is fast and disposable.

### Why Maven?

Maven is widely used in Java projects and makes it easy to manage Spring Boot dependencies, compile the project, execute tests, and package the application.

## Overall Application Flow

```text
User opens VoltPath
      |
      v
React landing page
      |
      v
User opens Trip Planner
```

```
        |
        v
Frontend loads vehicles and charging stations
        |
        v
User selects vehicle and origin/destination
        |
        v
React sends request to Spring Boot API
        |
        v
RoutingService calculates EV route
        |
        v
Backend returns route and charging stops
        |
        v
Frontend displays result and map markers
```

## Testing

The backend includes a JUnit test using Mockito. It tests the case where a route cannot be completed and no charging stations are available.

Relevant file: `RoutingServiceTest.java`

## Important Project Limitation

The current implementation uses:

- Vehicle range
- Connector type
- Charging station availability
- Geographic distance

It does **not yet use real-time traffic, road networks, elevation, charging duration, battery percentage, or external live charging-station APIs**. The route is based on geographic distance rather than actual driving roads.

A good short presentation description would be:

> VoltPath is a full-stack electric vehicle route-planning application built with React, Spring Boot, and PostgreSQL. It calculates range-aware EV journeys, identifies compatible charging stations, and displays the recommended route on an interactive map. The backend uses a Haversine-based greedy routing algorithm with a safety buffer to determine whether a vehicle can reach its destination safely.

RiskFlow

1. The Elevator Pitch (What is this project?)

**Project Name:** Loan Performance Intelligence Engine (GitHub: RiskFlow) **What it does:** It is an end-to-end Machine Learning system for structured finance (mortgages). It predicts whether a borrower will miss a payment (delinquency), default, or prepay their loan. It also flags fraudulent/erroneous

data from loan servicers, runs macroeconomic stress tests (what happens if interest rates spike?), and uses an LLM to generate plain-English risk reports for human reviewers.

2. The Tech Stack (Tools Used)

- **Core Data Science:** pandas, numpy
- **Predictive Modeling:** xgboost (Non-linear models), scikit-learn (Logistic Regression baseline, Isolation Forest for anomalies)
- **Survival Analysis:** lifelines (Kaplan-Meier curves, Cox Proportional Hazards)
- **Explainable AI (XAI):** shap (TreeExplainer for feature importance)
- **Generative AI / RAG:** faiss-cpu (Vector database), sentence-transformers (Text embeddings), openai SDK (Used as a universal client to call Gemini and Groq APIs)

3. Architecture & Code Structure (src/ folder)

If they ask how the code is organized, tell them you built it modularly so it could be deployed to production:

- config.py: The **single source of truth**. All column names, hyperparameters, and random seeds live here. (Judges love this because it prevents configuration drift).
- data_loader.py: Handles data ingestion. If no data exists, it dynamically generates a 500-loan synthetic dataset spanning several years so the pipeline never crashes.
- features.py: Feature engineering. Generates rolling Days Past Due (DPD) metrics and applies Target Encoding.
- train.py: Trains both XGBoost and Logistic Regression models. Calibrates their probabilities so 0.8 actually means 80% confidence.
- survival.py: Calculates "time-to-event" (how many months until a loan defaults).
- anomaly.py: Hybrid anomaly detection scoring.

- scenario.py: Modifies the features (e.g., bumps interest rates by 2%) and runs them back through the model to see portfolio stress.
- explain.py: Runs SHAP to prove the models aren't black boxes. Also runs Demographic Parity checks to ensure the model isn't biased against specific states or credit bands.
- llm_copilot.py: The automated reviewer.
- pipeline.py: The master orchestrator. Uses a custom _safe() wrapper function so if one step fails, the rest of the pipeline keeps running.

4. How to Answer the Hard Questions (The "Gotchas")

**Q: How did you prevent Data Leakage? (Crucial Question)**

*"I prevented leakage in three ways: First, I didn't use a random train/test split; I used a strict **chronological time-aware split** (70/15/15) so the model never trains on future data. Second, my Target Encoders were fitted strictly on the training set. Third, my rolling features (like 6-month average Days Past Due) used an expanding window with a .shift(1) offset, ensuring the model never peeked at the current month's target."*

**Q: Why did you calibrate the models, and how?**

*"Raw XGBoost outputs aren't true probabilities. I used **Platt Scaling (Sigmoid)** to calibrate the binary targets (like default/prepayment). But for multiclass targets (like next state transitions), Sigmoid doesn't work well because it assumes binary log-odds, so I used **Isotonic Regression**, which is non-parametric and fits multiclass distributions much better."*

**Q: How does your Anomaly Detection work?**

*"It's a hybrid system. Unsupervised ML (Isolation Forest) is great at finding hidden multivariate outliers, but it can miss simple logic errors. So, I combined it with a deterministic **Business Rule Engine** (e.g., flagging if current balance > original balance). The final anomaly score is an ensemble of both."*

**Q: What happens to the AI Copilot if the API goes down?**

*"I built a 3-tier graceful degradation system. The primary LLM is **Gemini 3.6 Flash**. If Google's API fails, it automatically falls back to **Groq (Qwen 27B)**. If there is no internet at all, it falls back to an **offline deterministic Python template** that injects the SHAP values and FAISS RAG contexts. The pipeline will never crash due to network failure."*

5. Your Final Metrics to Memorize

- **XGBoost Next 12m Default (ROC-AUC):** 0.998
- **XGBoost Next State Multiclass (Macro-F1):** 0.936
- **CoxPH Survival Concordance Index:** 0.703 (Baseline LogReg was 0.818)
- **Top Risk Driver (SHAP):** days_past_due

1. Executive Summary

**RiskFlow** is an end-to-end, ML-first Loan Performance Intelligence Engine. It moves beyond traditional "black-box" credit scoring by predicting multiple loan outcomes (delinquency, default, prepayment), isolating competing risks, stress-testing portfolios against macroeconomic shocks, and providing deep explainability. It culminates in a live Streamlit Command Center and a RAG-grounded LLM Copilot that acts as an assistant to human reviewers.

---

2. Core Tech Stack

- **Data Processing & Engineering:** Python, Pandas, NumPy
- **Predictive Modeling:** XGBoost, Scikit-Learn
- **Survival Analysis:** Lifelines
- **Explainability (XAI):** SHAP (SHapley Additive exPlanations)
- **LLM Copilot:** Gemini / Groq APIs (with RAG architecture)
- **Frontend UI:** Streamlit
- **Version Control & Deployment:** Git, GitHub, Streamlit Community Cloud

---

3. Advanced Features: What We Used & WHY We Used It

**1. Time-Aware Splitting (Chronological Validation)**

- **What we did:** Split the train and test data based on time (origination vintage and month) rather than random K-Fold splitting.
- **WHY we used it:** In finance, using random splits causes "future leakage" (the model learns from future economic conditions to predict the past). Time-aware splitting is the only mathematically rigorous way to prove a model will work in real life.

**2. Calibrated XGBoost & Logistic Regression Ensemble**

- **What we did:** Used XGBoost as the core predictive engine, but wrapped it in a CalibratedClassifierCV.
- **WHY we used it:** Loan defaults are highly imbalanced (e.g., 95% of people pay, 5% default). Standard models just output a 0 because it's statistically safe. XGBoost finds the complex nonlinear patterns, but we heavily calibrated the probabilities to ensure that if the model says there is a "15% chance of default," it actually means exactly 15%. (Measured by the Brier Score).

**3. Cause-Specific CoxPH Survival Models (Lifelines library)**

- **What we did:** Implemented a competing-risk survival model rather than a standard binary classifier.
- **WHY we used it:** Standard models ask *"Will this loan default?"* Survival models ask *"WHEN will it default?"* More importantly, a loan cannot default if it gets paid off early (prepayment). We used a Competing-Risk framework to isolate the Default curve from the Prepayment curve, preventing the math from getting confused.

**4. Monte Carlo Portfolio Simulation & Clustered Stress Testing**

- **What we did:** Sampled thousands of economic scenarios (HPI drops, unemployment spikes) and applied them to the loan portfolio.

- **WHY we used it:** We didn't just shift one variable in a vacuum (which is unrealistic). We used hierarchical clustering to group correlated features. This allows us to see how a macro-shock ripples through the entire portfolio and calculate the exact Value at Risk (VaR) under severe economic distress.

**5. Automated Counterfactual Explanations & SHAP**

- **What we did:** Extracted SHAP values for global/local importance, but took it a step further by calculating Counterfactuals.
- **WHY we used it:** SHAP tells you *why* a loan is risky (e.g., "LTV is too high"). Counterfactuals are actionable—they tell the reviewer exactly what needs to change to save the loan (e.g., "If you drop the balance ratio by 5%, the default probability falls below the 50% danger zone").

**6. Hybrid Anomaly Detection (Isolation Forest)**

- **What we did:** Combined unsupervised Machine Learning (Isolation Forests) with hard-coded, deterministic validation rules.
- **WHY we used it:** Validation rules catch obvious human errors (e.g., negative loan balances). Isolation Forests catch complex, statistical fraud (e.g., a loan that looks normal on paper, but has a weird combination of DTI and geography compared to its peers).

**7. Human-in-the-Loop Active Learning**

- **What we did:** Created a "Human Review Zone" pipeline that automatically queues any prediction with a 30-70% uncertainty score for manual review.
- **WHY we used it:** AI shouldn't make edge-case decisions alone. By queuing uncertain loans for human reviewers and logging their corrections, we created a continuous feedback loop. The model learns from human expertise and gets smarter over time.

**8. RAG-Grounded LLM Copilot**

- **What we did:** Hooked up an LLM (Gemini/Groq) using a Retrieval-

Augmented Generation (RAG) architecture restricted entirely to the project's data dictionary and validation rules.

- **WHY we used it:** LLMs hallucinate badly with raw financial numbers. By forcing the LLM to only read our SHAP drivers and our strict data dictionary, we guarantee it produces highly accurate, plain-English summaries that a human reviewer can trust without second-guessing.

**9. Streamlit Command Center (Live PSI Tracking)**

- **What we did:** Built a deployed, real-time dashboard tracking Population Stability Index (PSI) and model metrics.
- **WHY we used it:** A model is useless if it degrades silently in production. Tracking PSI instantly alerts the team if incoming loan demographics suddenly shift away from what the model was originally trained on, preventing catastrophic financial losses.